

Appendix

March 1, 2017

1 Demos

Demos of WEBRELATE can be found at <https://drive.google.com/open?id=0ByGcuXqa9Z3KVEhERjBmSXJRcOU>

2 Additional motivating examples

Example 1 [DBLP] *A user had a list of PL researchers and wanted to extract the number of papers they have published in PL conferences from DBLP (as shown in Figure 1).*

In this case, the example URL for Eran Yahav’s DBLP page is <http://dblp.uni-trier.de/pers/hd/y/Yahav:Eran> and WEBRELATE learns the URLs for the other authors in the spreadsheet. After learning the URLs, it extracts the required data from the “refine by venue” section of the web-page (Figure 1(b)). An entry in this section has the following format: `<button value=":facet:venue:PLDI">PLDI (12)</button>`. WEBRELATE learns an expression that returns the text in a button whose value attribute is derived from the **Conference** column in the input. This expression generates the entire string “PLDI (12)” and WEBRELATE learns another transformation program on top of this to extract only the number of papers (12) from the results.

	Author	Conference	# papers
1	Eran Yahav	PLDI	12
2	Swarat Chaudhuri	PLDI	5
3	Eran Yahav	POPL	4

(a)

refine by venue
PLDI (12)
SAS (10)
OOPSLA (7)
ISSTA (5)
VMCAI (4)
POPL (4)

(b)

Figure 1: Integrating a spreadsheet containing authors with the number of PL papers obtained from DBLP.

Example 2 Statistics about ATP players *The user has a list of ATP players and would like to get the number of singles won by the player in a particular*

year as shown in Figure 2. Example URL is http://www.espn.com/tennis/player/_/id/296/novak-djokovic

	Player	Year	# singles
1	Novak Djokovic	2016	7
2	Roger Federer	2009	4
3	Andy Murray	2012	3

Figure 2: Extracting number of singles won by a player in a particular year.

This URL also has strings like player id that are not part of the input, but our system can learn these URLs by leveraging the search results as before. There is, however, another challenge in learning this integration task—the data from the web-pages should be extracted based on the **year** in the input data. For example, Figure 3 shows the ESPN page for **Roger Federer**. As can be seen, this web-page lists all the statistics about the player in a tabular form sorted by the year. Our DSL for data extraction is powerful enough to learn a program that can extract a node in this web-page that is in the third column of a row that has the **year** string in the first column.

3 URL Learning Examples

Example 3 Consider another example where we want to learn the URL https://en.wikipedia.org/wiki/United_States from the input **United States** and the URL <https://en.wikipedia.org/wiki/India> from the input **India**. Figure 4(a) and (b) show portions of the DAGs for these two examples when using the first layer. Here, there is no common program that can learn both these examples together. In such situations, the layered search will move to the second layer. Figure 4(c) shows the extra edges added in layer 2. Now, these examples can be learned together as `Concat(ConstStr("https"), ..., ConstStr("/"), Replace(ConstPos(0), ConstPos(-1), " ", "-"))`.

www.espn.com/tennis/player/_/id/402/roger-federer

STATS				
YEAR	PRIZE MONEY	SINGLES TITLES	DOUBLES TITLES	SINGLES W-L
2016	\$1,527,349	0	0	22-7
2015	\$6,624,027	6	0	59-10
2014	\$9,380,122	5	0	73-12
2013	\$2,381,637	1	0	45-17
2012	\$5,594,942	6	0	71-12
2011	\$6,369,576	4	0	64-12
2010	\$7,698,289	5	0	65-13
2009	\$8,768,110	4	0	61-12
2008	\$5,896,679	1	1	66-15
2007	\$7,405,620	7	0	63-7
Career	\$98,830,825	68	8	1089-245

Figure 3: ESPN page for Roger Federer

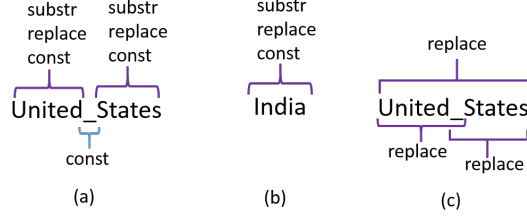


Figure 4: (a) and (b) shows a portion of the DAGs for Ex 3 using layer 1. (c) shows additional edges from layer 2.

4 Proofs

4.1 URL learning

Lemma 1 *The **GenProg** algorithm is sound for learning string expressions for all λ_s , λ_c and λ_a*

Proof: This holds because the **GenDag** algorithm only learns programs that are consistent for each input and **Intersect** preserves this consistency across multiple inputs.

Lemma 2 *The **GenProg** algorithm is complete for learning string expressions e when $\lambda_s = \text{True}$ and $\lambda_c = \text{True}$*

Proof: This is because when λ_s and λ_c are True, **GenDag** will learn all atomic expressions for every edge that satisfy the inputs. Since, the DAG structure allows all possible concatenations of atomic expressions, the **GenDag** algorithm is complete in this case. **Intersect** also preserves completeness and hence, **GenProg** is also complete.

Theorem 1 *Learning e expressions is sound i.e. given a few input-output examples $\{(v_i, o_i)\}_i$, the learnt program P will always satisfy $\forall i. \llbracket P \rrbracket_{v_i} = o_i$.*

Proof: This is because every layer in the layered search is sound using Lemma 1 and the transition between the layers is also sound.

Theorem 2 *Learning e programs is complete i.e. if there exists a string expression that satisfies the given input-output examples, then the algorithm is guaranteed to learn it given enough examples.*

Proof: This follows because the last layer in the layered search is complete using Lemma 2.

Theorem 3 *Learning filter programs is sound i.e. given some examples and a search query program, the learned filter is always consistent with the examples.*

Theorem 4 *Learning filter programs is complete in the limit $k \rightarrow \infty$ where k prefixes are stored for each node in the **SearchBestProg** algorithm.*

The proofs for these theorems follow because without **AnyStr** expressions, we have seen that learning e programs is sound and complete. **SearchBestProg** algorithm ensures soundness in the presence of **AnyStr** (for any k) because it only adds an atomic expression to a prefix if the combination is sound. Moreover, it also ensures completeness when $k \rightarrow \infty$, because in this limit, **SearchBestProg** will try all possible paths in the DAG to find a consistent program.

4.2 Data Extraction Learning

Lemma 3 *Any predicate expressed by the predicates graph for an example (i_k, w_k, γ_k) will satisfy the example.*

Proof: This lemma is true because **LearnTarget** constructs predicates graph by only adding those nodes and predicates that are in the original HTML document.

Lemma 4 *The predicates graph for an example (i_k, w_k, γ_k) can express all predicates in L_w that satisfy the example.*

Proof: This lemma is true because the traversal of nodes when constructing the predicates graph covers all possible paths in the HTML document that can be expressed in L_w .

Lemma 5 *Any predicate obtained after refining a path satisfies all the examples.*

Proof: This holds because the **IntersectPath** only adds those node predicates to the path that can be satisfied by the examples.

Lemma 6 *Refining a path preserves all predicates that satisfy the examples.*

Proof: This holds because the algorithm for refining a path intersects the path with all similar paths in the other examples and **IntersectPath** preserves all predicates that can be satisfied by all the examples.

Theorem 5 *The data extraction synthesis algorithm is sound i.e. given some input-output examples $\{(i_k, w_k, \gamma_k)\}_k$, the program P that is learned by the algorithm will always satisfy $\forall k. \llbracket P \rrbracket_{i_k}(w_k) = \{\gamma_k\}$.*

Proof: This theorem follows from Lemma 3 and 5.

Theorem 6 (Completeness) *The data extraction synthesis algorithm is complete i.e. if there exists a program in L_w that satisfies the given input-output examples, then the algorithm is guaranteed to find one program that satisfies the examples.*

Proof: This theorem follows from Lemma 4 and 6.

Note, that the **SearchBestProg** algorithm cannot influence the soundness or completeness argument because the set of all predicates obtained after refining every path in the predicates graph is a sound and complete solution. **SearchBestProg** only influences how well it generalizes to unseen inputs.

5 Additional results

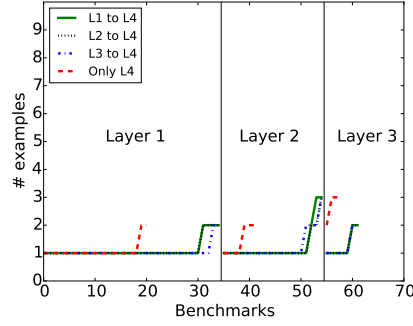


Figure 5: Graph showing number of examples required for learning URLs using search engine for the different configurations of layered search.

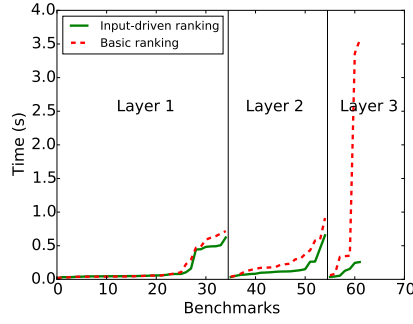


Figure 6: Graph showing synthesis times for learning URLs using search query with and without input-driven ranking.

6 Benchmarks

Table 1 and Table 2 lists all our benchmarks along with a brief description to the integration task, # of input rows, and an example each for the desired data and URL.

#	Description	# rows	Example data
1	ATP players to ages	5	(age 29)
2	ATP players to latest tweet	5	Long text
3	ATP players to number of singles based on year	20	7
4	ATP players to number of singles in 2016	7	7
5	ATP players to number of tweets	20	2,324
6	ATP players to rankings	20	ATP Rank #1

7	ATP players to rankings	5	1
8	ATP players to rankings	5	#1
9	ATP players to rankings	5	66 (7th in the Open Era)
10	ATP players to singles W-L	5	742-152
11	Address to weather based on date	6	62
12	Addresses to population	7	668,342 (100% urban, 0% rural).
13	Addresses to population	7	786,130
14	Addresses to weather	7	51°
15	Addresses to weather	7	57 F
16	Addresses to weather	7	52
17	Addresses to weather based on date	6	49
18	Addresses to weather based on date	6	Partly sunny
19	Addresses to weather based on date	6	74°
20	Addresses to weather based on date	7	51°
21	Addresses to weather based on the date	6	68°
22	Addresses to zipcode	7	98101 Zip Code
23	Airport code to airport name	5	Boston Logan International Airport (BOS)
24	Airport code to delay status	5	Very few delays
25	Airport code to terminal information	5	Long text
26	Airport codes to airport names	5	Long text
27	Albums to genre	5	Rock
28	Authors to citations based on paper title	5	Cited by 175
29	Authors to highest citations	5	Cited by 427
30	Authors to latest paper from DBLP	5	A differential approach to undefined behavior detection.
31	Authors to most cited papers	5	Hoard: A scalable memory allocator for multithreaded applications
32	Authors to most co-authored person	5	Alvin Cheung (12)
33	Authors to number of PLDI papers	5	PLDI (9)
34	Authors to number of papers based on conference	10	PLDI (9)
35	Cities to population	6	21,357,000
36	Company symbols to 1 year target prices	6	65
37	Company symbols to stock prices	6	59.87
38	Company symbols to stock prices	6	59.87
39	Company symbols to stock prices	6	59.87
40	Company symbols to stock prices	6	59.87
41	Company symbols to stock prices	6	59.00
42	Company symbols to stock prices	6	59.95
43	Company symbols to stock prices	6	\$59.87
44	Company symbols to stock prices based on date	6	60.61
45	Company symbols to stock prices based on date	6	60.61
46	Country names to population	6	1,326,801,576
47	Country names to population	6	1,336,286,256

48	Cricket results for teams on different dates	5	Australia won by 3 wickets (with 19 balls remaining)
49	Cricket stats (total matches) for two different teams	5	90
50	Cricket stats (won) for two different teams	5	24
51	Cricket stats(average) for two different teams	5	31.67
52	Currency exchange values	8	66.7870
53	Currency exchange values	8	INR/USD = 66.84699
54	Currency exchange values	8	66.7800
55	Currency exchange values	8	66.8043 INR
56	Currency exchange values	8	66.779
57	Currency exchange values	8	66.7914
58	Currency exchange values based on date	8	64.1643 INR
59	Currency exchange values based on date	8	66.778
60	Currency exchange values based on date	8	64.2308481698
61	Flight distance between two cities	5	2,487 Miles
62	Flight fares (cheapest day) between two cities	5	Wednesday
63	Flight fares between two cities	5	\$257
64	Flight fares between two cities	5	\$217
65	Flight fares between two cities based on airlines	5	\$237
66	Flight travel time between two cities	5	6 hrs 11 mins
67	Flight travel time between two cities	5	5 hours, 38 minutes
68	NFL teams to rankings	5	4-5, 3rd in AFC East
69	NFL teams to rankings	5	26th
70	NFL teams to rankings	5	26.7
71	NFL teams to rankings	5	#5
72	NFL teams to stadium names	32	New Era Field
73	Nobel prize for different subjects based on year	5	Richard F. Heck
74	Nobel prize for different years based on subject	5	The Nobel Prize in Chemistry 2010 was awarded jointly to Richard F. Heck, Ei-ichi Negishi and Akira Suzuki "for palladium-catalyzed cross couplings in organic synthesis".
75	Nobel prize for different years based on subject	5	Richard F. Heck, Ei-ichi Negishi and Akira Suzuki
76	Novels to authors	7	Charles Dickens
77	Novels to authors	8	Charles Dickens
78	Novels to genre	7	Historical novel
79	Number of coauthored papers between two authors	10	Rastislav Bodk (6)
80	Number of daily flights between two cities	5	26
81	People to profession	5	West Higher Education, Public Sector Healthcare, State and Local Govt. Business Development Director
82	Real estate properties to sale prices	5	\$1,452
83	Real estate properties to stats	5	1,804 sqft

84	Stock prices based on company names from same webpage	6	0.00
85	Videos to number of likes	5	11,473,055
86	Videos to number of youtube views	5	2,672,818,718 views
87	Videos to publish date	5	Published on Jul 15, 2012
88	ATP players to countries	5	Serbia and Montenegro

Table 1: Benchmarks description along with number of input rows and example data that is extracted.

#	Example URL
1	https://en.wikipedia.org/wiki/Novak_Djokovic
2	https://twitter.com/djokernole
3	http://www.espn.com/tennis/player/_/id/296/novak-djokovic
4	http://espn.go.com/tennis/player/_/id/296/novak-djokovic
5	https://twitter.com/djokernole
6	http://espn.go.com/tennis/player/_/id/296/novak-djokovic
7	http://www.atpworldtour.com/en/players/novak-djokovic/d643/overview
8	http://www.tennis.com/player/532/novak-djokovic/
9	https://en.wikipedia.org/wiki/Novak_Djokovic
10	http://espn.go.com/tennis/player/_/id/296/novak-djokovic
11	http://www.accuweather.com/en/us/seattle-wa/98104/september-weather/351409
12	http://www.city-data.com/city/Seattle-Washington.html
13	http://zipcode.org/city/WA/SEATTLE
14	http://www.accuweather.com/en/us/seattle-wa/98104/weather-forecast/351409
15	http://www.timeanddate.com/weather/usa/seattle
16	https://weather.com/weather/today/1/Seattle+WA+98109:4:US#!
17	http://www.accuweather.com/en/us/seattle-wa/98104/december-weather/351409
18	http://www.accuweather.com/en/us/seattle-wa/98104/december-weather/351409
19	http://www.accuweather.com/en/us/seattle-wa/98104/september-weather/351409
20	http://www.accuweather.com/en/us/seattle-wa/98104/weather-forecast/351409
21	http://www.accuweather.com/en/us/seattle-wa/98104/september-weather/351409
22	http://zipcode.org/city/WA/SEATTLE
23	https://www.virginamerica.com/cms/fly-with-us/airport-info/boston-bos/
24	http://www.flightstats.com/go/Airport/airportDetails.do;airportCode=BOS
25	https://www.aa.com/i18n/travelInformation/destinationInformation/bos-airport.jsp
26	http://www.flightstats.com/go/Airport/airportDetails.do;airportCode=BOS
27	https://en.wikipedia.org/wiki/The_Joshua_Tree
28	https://scholar.google.com/scholar?q=emery+berger
29	https://scholar.google.com/scholar?q=emery+berger
30	http://dblp.uni-trier.de/pers/s/Solar=Lezama:Armando
31	https://scholar.google.com/scholar?q=emery+berger
32	http://dblp.uni-trier.de/pers/s/Solar=Lezama:Armando
33	http://dblp.uni-trier.de/pers/s/Solar=Lezama:Armando
34	http://dblp.uni-trier.de/pers/s/Solar=Lezama:Armando

35 <http://worldpopulationreview.com/world-cities/mumbai-population/>
36 <http://www.nasdaq.com/symbol/msft>
37 <http://finance.yahoo.com/q?s=MSFT>
38 <http://money.cnn.com/quote/quote.html?symb=MSFT>
39 <http://quotes.wsj.com/MSFT>
40 <http://www.google.com/finance?cid=358464>
41 <http://www.marketwatch.com/investing/stock/MSFT/historical>
42 <http://www.marketwatch.com/investing/stock/msft>
43 <http://www.nasdaq.com/symbol/msft>
44 <http://www.google.com/finance/historical?q=NASDAQ:MSFT>
45 <http://www.google.com/finance/historical?q=NASDAQ:MSFT>
46 <http://www.worldometers.info/world-population/india-population/>
47 https://en.wikipedia.org/wiki/Demographics_of_India
48 <http://www.espn.com/ci/engine/match/index.html?date=2016-08-21>
49 <http://stats.espn.com/ci/engine/team/6.html?class=1;template=results;type=team>
50 <http://stats.espn.com/ci/engine/team/6.html?class=1;template=results;type=team>
51 <http://stats.espn.com/ci/engine/team/6.html?class=1;template=results;type=team>
52 <http://finance.yahoo.com/echarts?s=USDINR=X>
53 <http://thefinancialconverter.com/USD/INR.aspx>
54 <http://www.bloomberg.com/quote/USDINR:CUR>
55 <http://www.google.com/finance?q=USDINR>
56 <http://www.investing.com/currencies/usd-inr>
57 <http://www.xe.com/currencyconverter/convert/?Amount=1&From=USD&To=INR>
58 <http://www.exchange-rates.org/Rate/USD/INR/6-16-2015>
59 <http://www.investing.com/currencies/usd-inr-historical-data>
60 <http://www.xe.com/currencytables/?from=USD&date=2015-06-16>
61 <https://www.cheapoair.com/flights/cheap-flights-from-boston-to-seattle-bos-sea>
62 <http://www.farecompare.com/flights/Boston-BOS/Seattle-SEA/market.html>
63 <http://www.cheapflights.com/flights-to-seattle/boston/>
64 <http://www.farecompare.com/flights/Boston-BOS/Seattle-SEA/market.html>
65 <http://www.farecompare.com/flights/Boston-BOS/Seattle-SEA/market.html>
66 <http://www.cheapflights.com/flights-to-seattle/boston/>
67 <http://www.travelmath.com/flying-time/from/Boston,+MA/to/Seattle,+WA>
68 http://espn.go.com/nfl/team/_/name/buf/buffalo-bills
69 <http://www.cbssports.com/nfl/teams/page/BUF/buffalo-bills>
70 <http://www.nfl.com/teams/buffalobills/profile?team=BUF>
71 <https://www.teamrankings.com/nfl/team/buffalo-bills>
72 http://espn.go.com/nfl/team/stadium/_/name/buf/buffalo-bills
73 https://en.wikipedia.org/wiki/List_of_Nobel_laureates_in_Chemistry
74 <http://www.nobelprize.org/nobel-prizes/chemistry/laureates/2010/>
75 <http://www.nobelprize.org/nobel-prizes/lists/year/?year=2010>
76 https://en.wikipedia.org/wiki/A_Tale_of_Two_Cities
77 https://www.goodreads.com/book/show/1953.A_Tale_of_Two_Cities
78 https://en.wikipedia.org/wiki/A_Tale_of_Two_Cities
79 <http://dblp.uni-trier.de/pers/s/Solar=Lezama:Armando>
80 <https://www.cheapoair.com/flights/cheap-flights-from-boston-to-seattle-bos-sea>

81	http://www.zoominfo.com/p/Tom-Ball/14114519
82	http://www.zillow.com/homedetails/512-213th-St-SW-Bothell-WA-98021/38563217_zpid/
83	http://www.zillow.com/homedetails/512-213th-St-SW-Bothell-WA-98021/38563217_zpid/
84	http://www.marketwatch.com/tools/industry/stocklist.asp?bcind_ind=9535&bcind_period=3mo
85	http://www.youtube.com/watch?v=9bZkp7q19f0
86	http://www.youtube.com/watch?v=9bZkp7q19f0
87	http://www.youtube.com/watch?v=9bZkp7q19f0
88	https://en.wikipedia.org/wiki/Novak_Djokovic

Table 2: Benchmarks cont: Example URL for each benchmark